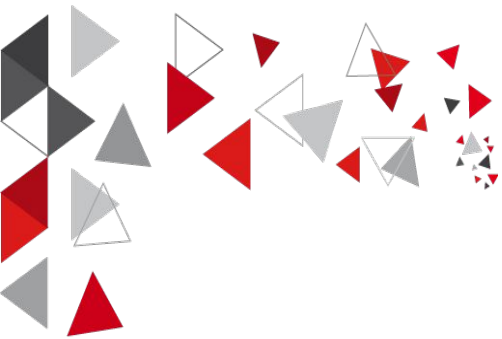
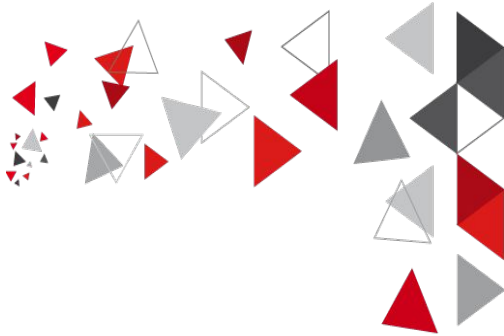


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

Chapitre 4: Encapsulation

Objectifs du chapitre



- Notion de package
- Encapsulation des classes
- Encapsulation des attributs
- Encapsulation des méthodes

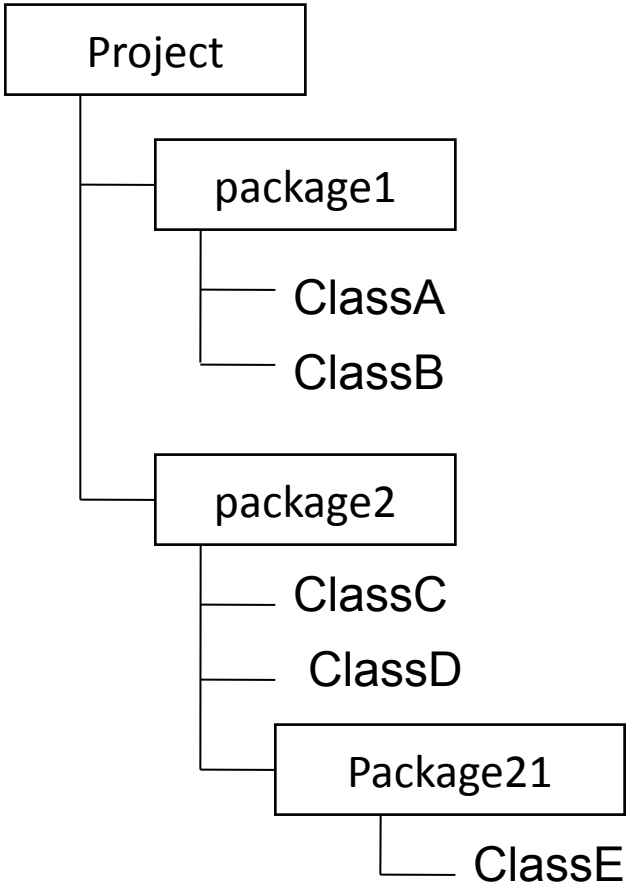
Package

Les packages en Java sont un mécanisme de regroupement de classes qui permet de structurer les fichiers de votre projet en catégories logiques.

Les paquetages (packages) sont organisés hiérarchiquement comme des répertoires de classes.

⇒ Package = répertoire.

Package



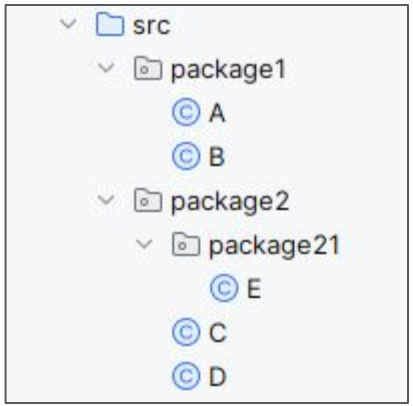
Déclaration d'un package

```
package package1;
```

Import d'un package

```
import package2.ClassC; //importer une seule Classe

import package2.*; //importer toutes les Classes sous le package
```

Structure de projet



Déclaration d'un package

A.java

```
package package1;

public class A {
}
```

Import d'un package

A.java

```
package package1;

import package2.C;

public class A {

    C  c = new C();
}
```

Il existe également des packages qui sont automatiquement importés pour chaque classe Java d'une façon implicite.

Ex:
Les classes du package “java.lang” (String, Integer, Boolean,Object ….).

Il existe un package par défaut en Java, appelé "default package", qui est utilisé pour les classes qui ne sont pas explicitement déclarées dans un package. Ces classes ne sont accessibles que par celles qui se trouvent dans le même package par défaut.

Encapsulation

L'encapsulation est un concept qui consiste à rendre les membres d'un objet **plus ou moins visibles** pour les autres objets.

L'encapsulation est une manière de définir une classe de telle sorte que ses attributs ne puissent pas **être directement manipulés de l'extérieur** de la classe, mais seulement indirectement par **l'intermédiaire des méthodes.**

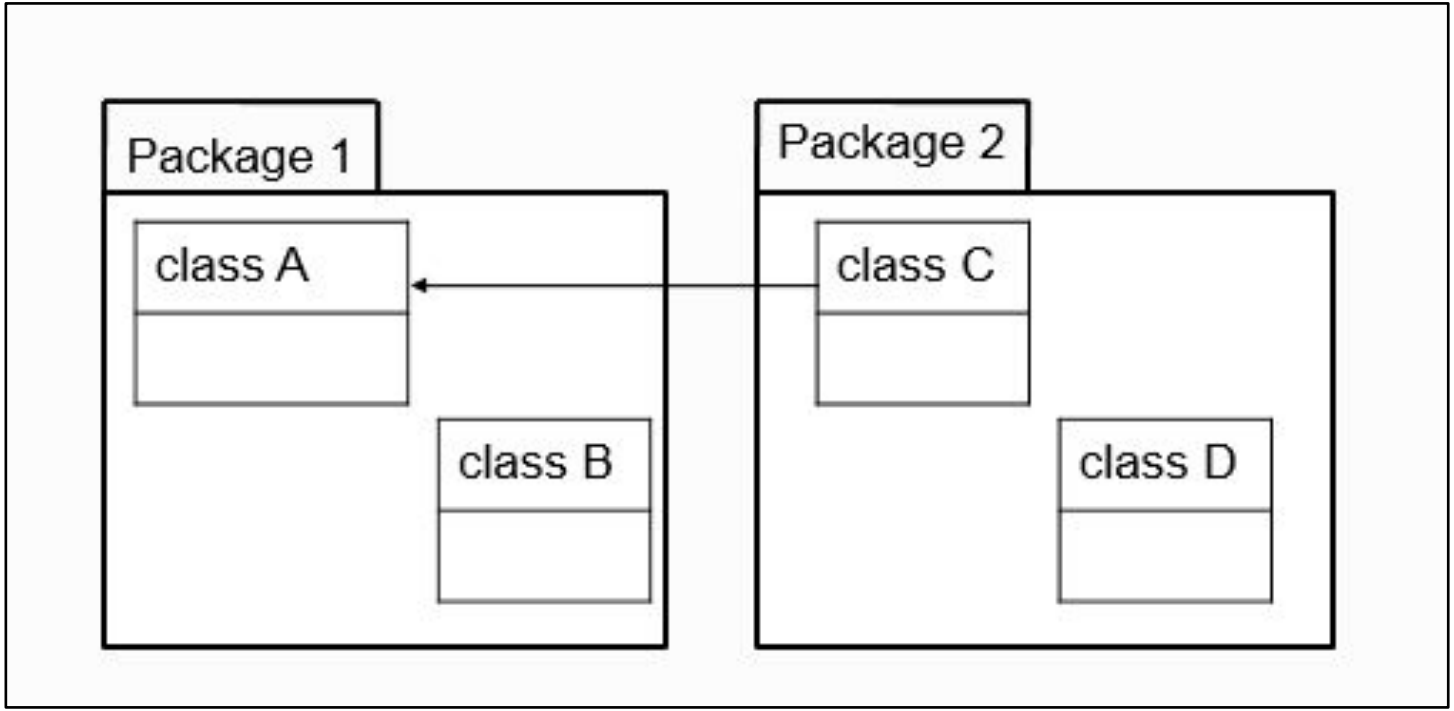
L'encapsulation peut être appliquée à différents éléments, notamment :

- Les classes : vous pouvez contrôler qui peut accéder aux classes.
- Les constructeurs : vous pouvez contrôler qui peut instancier les classes.
- Les variables d'instance : vous pouvez contrôler qui peut accéder aux variables d'instance de votre classe.
- Les méthodes : vous pouvez contrôler qui peut appeler les méthodes de votre classe.

Java offre 4 modificateurs d’accès:

- public : accès non contrôlé
- private : accès limité à la classes courante
- protected : accès limité au membre du package et à ses descendants (classes filles)
- default : accès limité au membre du package. Il n'existe pas de mot clef pour ce type de modificateur, de ce fait si aucun mot clé parmi public, private ou protected n'est utilisé alors l'élément est considéré comme étant par défaut.

Encapsulation des classes (1/3)

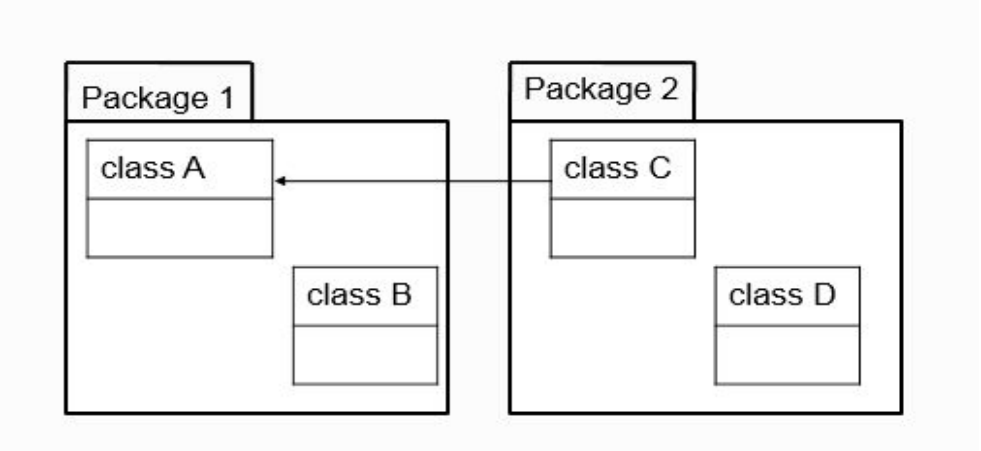


Les classes peuvent uniquement être publiques ou par défaut.

Encapsulation des classes (2/3)

public

```
public class A {
…
}
```




```
class B {
      A a;
…
}
```

```
class C  extends A {
      A a;
…
}
```

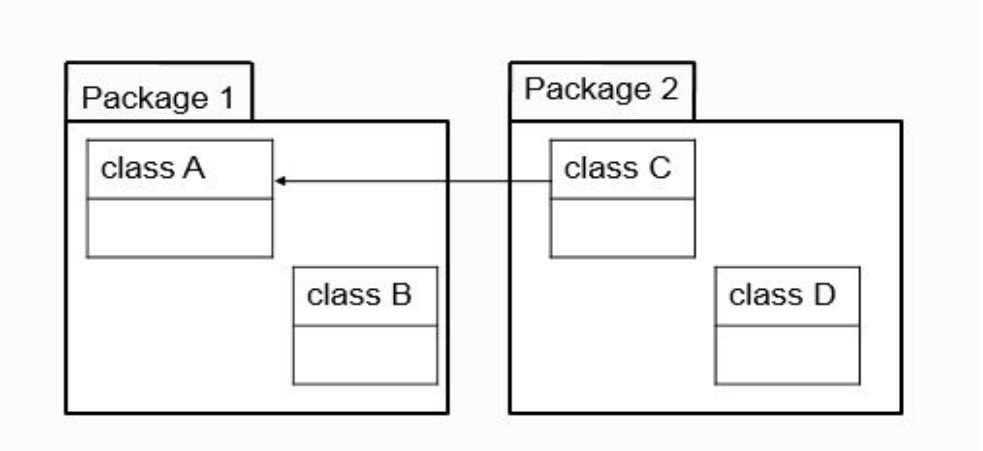
```
class D {
      A a;
…
}
```

Une classe **public** est visible depuis n'importe quelle classe du projet.

Encapsulation des classes (3/3)

default

```
class A {
…
}
```




```
class B {
      A a;
…
}
```

```
class C  extends A {
      A a;
…
}
```

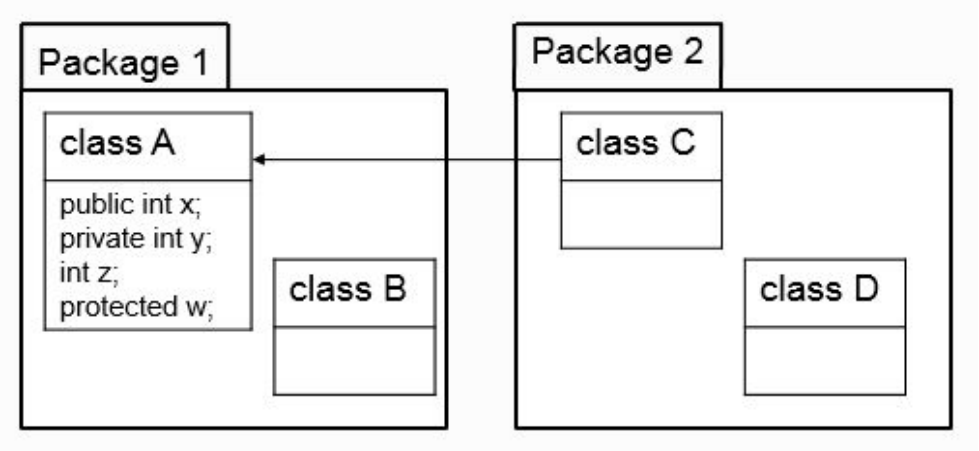
```
class D {
      A a;
…
}
```

Une classe **default** est visible seulement par les classes de son package.

Encapsulation des attributs (1/4)

public

```
public class A {
public int x;
…
}
```




```
class B {
 A a=new A();
 a.x = t ;

}
```

```
class C  extends A {
     A a=new A();
     a.x = t ;
     this.x = t ;
}
```

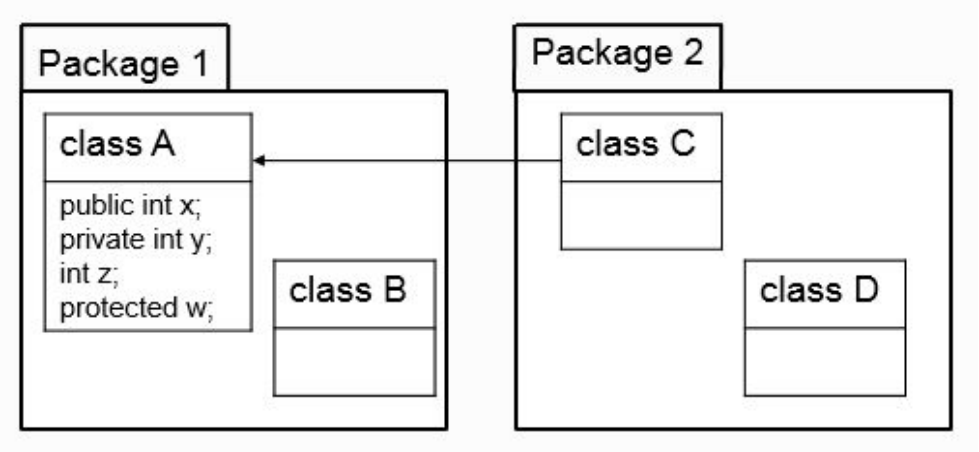
```
class D {
     A a=new A();
     a.x = t ;

}
```

Un attribut **public** est visible par toutes les classes

Encapsulation des attributs (2/4)

private

```
public class A {
private int y;
…
}
```




```
class B {
 A a=new A();
 a. y = t;

}
```

```
class C  extends A {
    A a=new A();
    a. y = t;
    y= t;
}
```

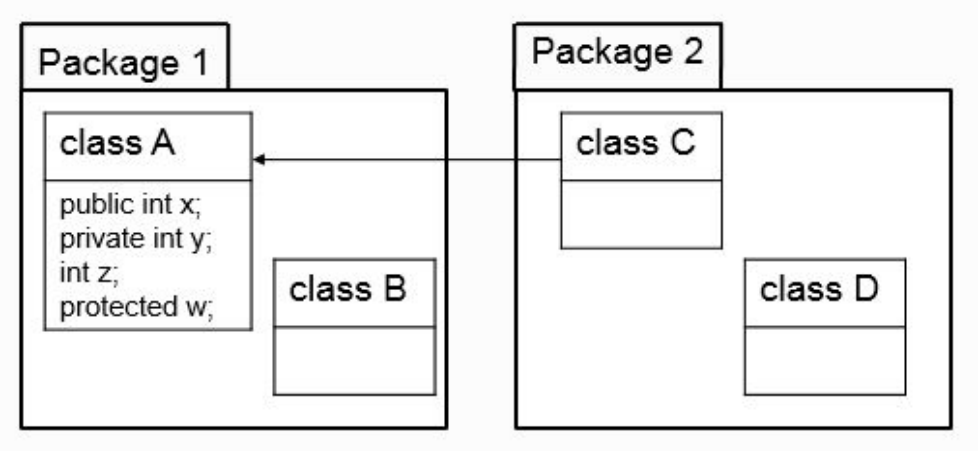
```
class D {
    A a=new A();
    a. y = t;

}
```

Un attribut **private** n'est accessible que depuis l'intérieur même de la classe.

Encapsulation des attributs (3/4)

default

```
public class A {
int z;
…
}
```




```
class B {
 A a=new A();
 a.z = t;

}
```

```
class C  extends A {
     A a=new A();
     a.z = t;
     z= t;
}
```

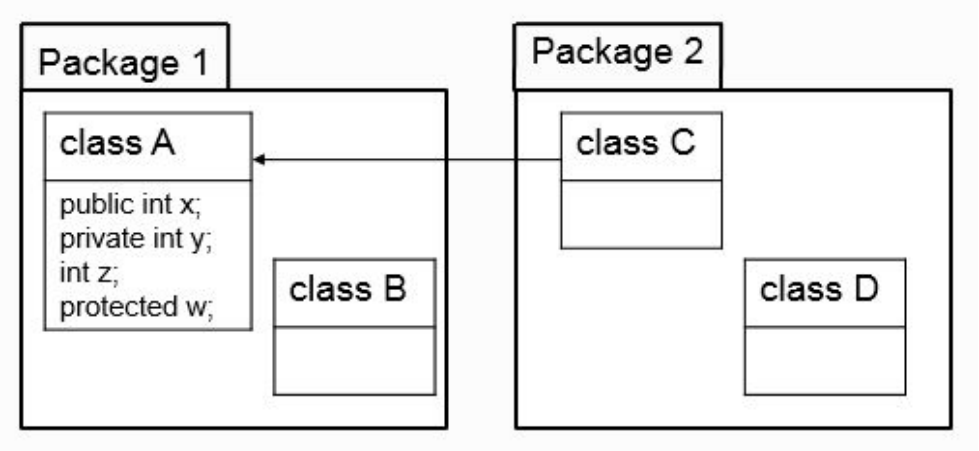
```
class D {
     A a=new A();
     a.z = t;

}
```

La variable **default** n'est accessible que depuis les classes du même package.

Encapsulation des attributs (4/4)

protected

```
public class A {
protected int w;
…
}
```




```
class B {
 A a=new A();
 a.w = t;

}
```

```
class C  extends A {
    A a=new A();
    a.w = t;
    this.w= t;
}
```

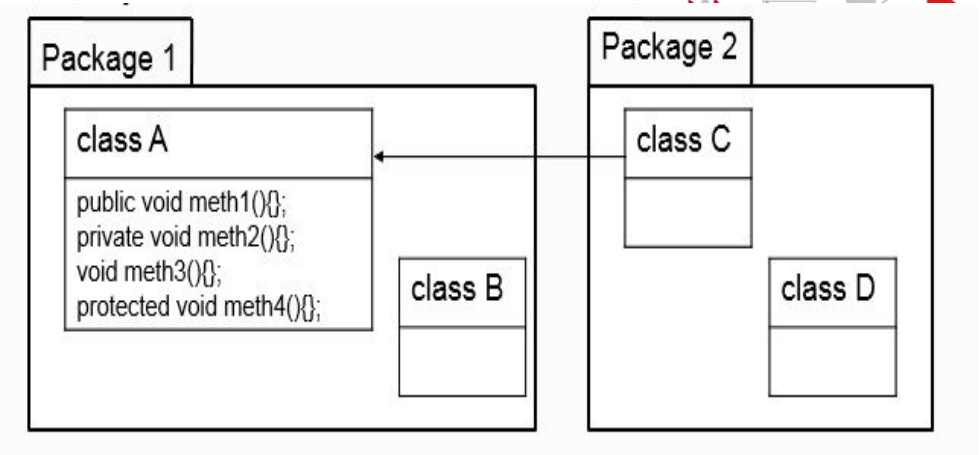
```
class D {
    A a=new A();
    a.w = t;

}
```

Un attribut **protected** est accessible uniquement aux classes du même package et à ses sous-classes (même si elles sont définies dans un package différent.)

Encapsulation des méthodes(1/4)

public

```
public class A {
public void meth1(){   }
…
}
```




```
class B {
 A a=new A();
 a.meth1() ;

}
```

```
class C  extends A {
    A a=new A();
    a.meth1() ;
    this.meth1();
}
```

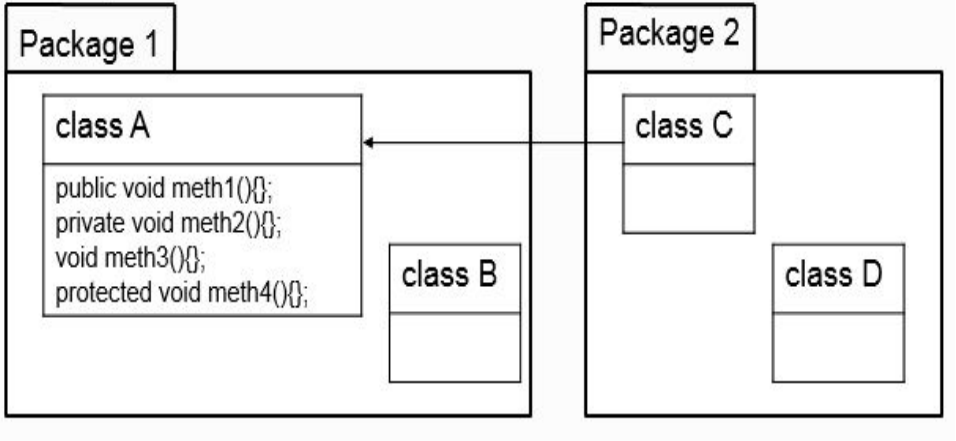
```
class D {
    A a=new A();
    a.meth1() ;

}
```

Une méthode **public** est visible par toutes les classes

Encapsulation des méthodes(2/4)

private

```
public class A {
private void meth2(){   }
…
}
```




```
class B {
 A a=new A();
 a.meth2() ;

}
```

```
class C  extends A {
   A a=new A();
   a.meth2();
   meth2();
}
```

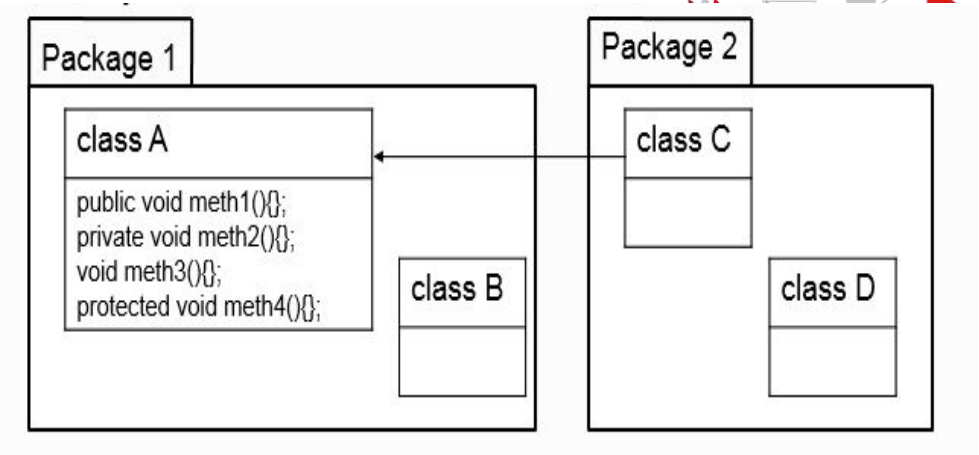
```
class D {
 A a=new A();
 a.meth2() ;

}
```

Une méthode **private** n'est accessible que depuis l'intérieur de sa classe.

Encapsulation des méthodes(3/4)

default

```
public class A {
void meth3(){   }
…
}
```




```
class B {
 A a=new A();
 a.meth3() ;

}
```

```
class C  extends A {
   A a=new A();
   a.meth3();
   meth3();
}
```

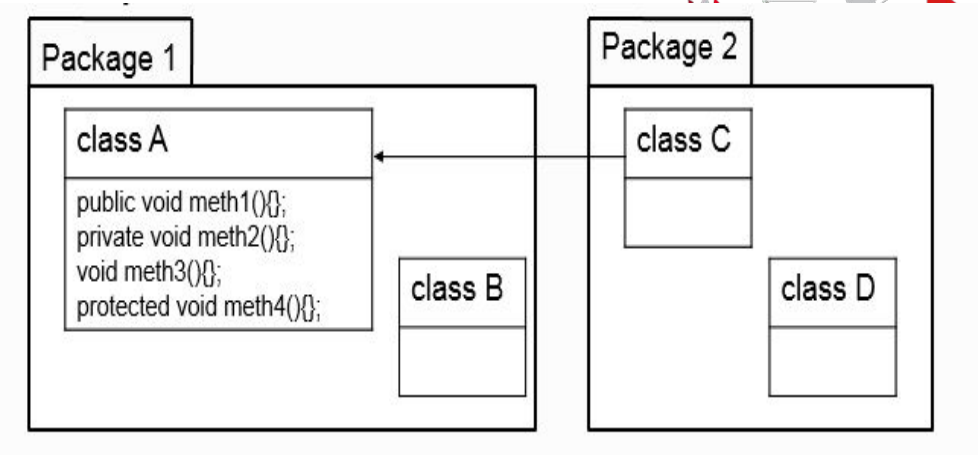
```
class D {
 A a=new A();
 a.meth3() ;

}
```

Une méthode **default** n'est accessible que depuis les classes du même package.

Encapsulation des méthodes(4/4)

protected

```
public class A {
protected void meth3(){   }
…
}
```




```
class B {
 A a=new A();
 a.meth4() ;

}
```

```
class C  extends A {
   A a=new A();
   a.meth4();
   this.meth4();
}
```

```
class D {
 A a=new A();
 a.meth4();

}
```

Une méthode **protected** est accessible aux classes du même package et à ses sous-classes (même si elles sont définies dans un package différent.)

Encapsulation des méthodes/attributs

Modificateur	Classe	Paquetage	Classe fille	Autre classe
private	visible			
default	visible	visible (même package)	visible* (même package)	
protected	visible	visible	visible	
public	visible	visible	visible	visible

*Les sous-classes peuvent accéder aux attributs et aux méthodes de la classe mère si elles se trouvent dans le même package

Les accesseurs (GET) et les mutateurs (SET)

Les accesseurs ("getters") et les mutateurs ("setters") sont des méthodes spéciales qui permettent d'accéder et de modifier les attributs d'un objet.

Les accesseurs sont utilisés pour lire la valeur d'une propriété, tandis que les mutateurs sont utilisés pour en changer la valeur.

Attention : Les setter et les getter doivent être déclarés **public**

Les accesseurs (GET) et les mutateurs (SET)

Les accesseurs sont généralement définis avec le mot-clé "get" suivi du nom de la propriété, par exemple :

```
public class Product {
	private int quantity;

	public int getQuantity() {
		return quantity;
	}
}
```

Les mutateurs sont généralement définis avec le mot-clé "set" suivi du nom de la propriété, par exemple :

```
public class Product {
	private int quantity;

	public void setQuantity(int quantity) {
		this.quantity = quantity;
	}
}
```

Les accesseurs (GET) et les mutateurs (SET)

L’accesseur d'un attribut booléen (boolean) s'écrit avec le "is",

exemple :

```
public class Product {
	private boolean onSale;

	public boolean isOnSale() {
		return onSale;
	}
}
```

Les accesseurs (GET) et les mutateurs (SET)

L'utilisation des getter et setter permet de gérer les modifications de manière centralisée et de valider les entrées avant de les enregistrer.

```
public class Product {
	private int quantity;

	public void setQuantity(int quantity) {
		if (quantity < 0)
			System.out.println("The quantity must be a positive number");
		else
			this.quantity = quantity;
	}
}
```

Merci pour votre attention